Studio Nx

JS 306 — 오디오 회귀 테스트 실행 도구

2026-10-21

JavaScript 306 — 실행 도구 (`runner.mjs`, `plugins.mjs`, `wavio.mjs`)

항목	내용
파일	`studionx/tests/audio/lib/runner.mjs`, `studionx/tests/audio/lib/plugins.mjs`, `studionx/tests/audio/lib/wavio.mjs`
작업	S1-12 오디오 회귀 테스트 기반: 테스트 프로젝트를 앱의 `renderOffline`으로 렌더링해 오는 도구 (PRD 00 §7.1)
작성	Claude (AI), 2026-10-21, 사용자 검토
이전, 다음	이전 305 비교 지표, 다음 307 실행 명령과 자체 시험

1. 하는 일

서버(FastAPI)와 DB 없이, 앱의 소리 코드를 그대로 헤드리스 Chrome에서 돌려 테스트 프로젝트의 소리를 얻어옵니다.

```
Node (run.mjs)                          헤드리스 Chrome
  startServer: frontend 폴더를 내줌  ─▶  /__harness.html 을 엽니다
  buildArgs: 프로젝트 JSON → tick 단위의 인자
  render(인자) ──────────────────────▶  시험용 소리를 수식으로 만들고 renderOffline(앱의 코드)을 부름
  채널별 Float32Array 로 받음 ◀──────  결과를 base64 로 돌려줌
  compareRenders(305)로 골든과 비교
```

파일	맡은 일
`runner.mjs`	정적 서버, Chrome 실행과 DevTools 연결, 렌더링 호출, 프로젝트 JSON 변환
`plugins.mjs`	플러그인 정의(`param_schema`)를 DB 시드 SQL에서 읽기
`wavio.mjs`	골든 WAV를 32bit float로 쓰고 읽기

2. 언제 쓰나

쓰는 곳	하는 일
`run.mjs` (307)	`openRenderer`로 렌더링기를 열고, 프로젝트마다 `buildArgs`와 `render`를 부릅니다. 골든은 `readWav`와 `writeFloatWav`로 다룹니다.
`selftest.mjs` (307)	같은 도구로 프로젝트를 일부러 바꿔 렌더링해 실패하는지 확인합니다.

3. 코드

3.1 프로젝트 JSON 변환과 골든 경로

```
export function loadProjects(only) {
  return fs.readdirSync(PROJECTS_DIR)
    .filter(function (f) { return f.endsWith('.json'); })
    .sort()
    .map(function (f) { return JSON.parse(fs.readFileSync(path.join(PROJECTS_DIR, f), 'utf-8')); })
```

```
    .filter(function (p) { return !only || only.indexOf(p.name) >= 0; });
}
```

```
export function buildArgs(p, schemas) {
  const sch = schemas || loadPluginSchemas();
  const need = function (code) {
    if (!sch[code]) throw new Error('프로젝트 ' + p.name + ': 알 수 없는 플러그인 ' + code);
    return sch[code];
  };
  const tracks = (p.tracks || []).map(function (t) {
    return { track_id: t.id, track_type: t.type, volume_db: t.volume_db || 0, pan: t.pan || 0, is_muted:
!!t.mute, is_solo: !!t.solo };
  });
  if (!tracks.some(function (t) { return t.track_type === 'MASTER'; })) tracks.push({ track_id: 999, tra
ck_type: 'MASTER', volume_db: p.master_db || 0, pan: 0, is_muted: false, is_solo: false });

  const instruments = {};
  Object.keys(p.instruments || {}).forEach(function (tid) {
    instruments[tid] = p.instruments[tid].map(function (l, i) {
      return { id: Number(tid) * 100 + i + 1, plugin_code: l.plugin, schema: need(l.plugin), params: l.p
arams || {}, volume_db: l.volume_db || 0, pan: l.pan || 0, is_enabled: l.enabled !== false };
    });
  });
  const effects = {};
  Object.keys(p.effects || {}).forEach(function (tid) {
    effects[tid] = p.effects[tid].map(function (e, i) {
      return { id: Number(tid) * 100 + i + 1, plugin_code: e.plugin, schema: need(e.plugin), params: e.p
arams || {}, is_bypassed: !!e.bypass, slot_index: i };
    });
  });
  const audioClips = (p.audioClips || []).map(function (c, i) {
    return {
      clip_id: i + 1, track_id: c.track, sample_id: c.sample,
      start_tick: Math.round(c.startBar * TICKS_PER_BAR), length_tick: Math.round(c.lengthBars * TICKS_P
ER_BAR),
      offset_tick: Math.round((c.offsetBars || 0) * TICKS_PER_BAR), gain_db: c.gain_db || 0, is_looped:
!!c.loop, pitch_semitones: c.pitch || 0,
    };
  });
  const midiClips = (p.midiClips || []).map(function (c) {
    return {
      track_id: c.track, start_tick: Math.round(c.startBar * TICKS_PER_BAR), length_tick: Math.round(c.l
engthBars * TICKS_PER_BAR), offset_tick: 0,
      notes: c.notes.map(function (n) {
        return { pitch: n.pitch, velocity: n.velocity == null ? 100 : n.velocity, start_tick: Math.round
(n.startBeat * TICKS_PER_BEAT), length_tick: Math.round(n.lengthBeat * TICKS_PER_BEAT) };
      }),
    };
  });
  return {
    samples: p.samples || {}, tracks: tracks, instruments: instruments, effects: effects, audioClips: au
dioClips, midiClips: midiClips,
    bpm: p.bpm || 120,
    tempoChanges: (p.tempoChanges || []).map(function (c) { return { tick: Math.round(c.bar * TICKS_PER_
BAR), bpm: c.bpm, ramp: c.ramp || 'STEP' }; }),
    startTick: Math.round((p.startBar || 0) * TICKS_PER_BAR),
```

```
    endTick: Math.round(p.endBar * TICKS_PER_BAR), tailSec: p.tailSec || 0, sampleRate: p.sampleRate ||
44100,
  };
}
```

```
export function goldenPath(name) {
  return path.join(GOLDEN_DIR, name + '.wav');
}
```

3.2 서버, 브라우저, 렌더링

```
export function startServer() {
  return new Promise(function (resolve) {
    const server = http.createServer(function (req, res) {
      const url = decodeURIComponent((req.url || '/').split('?')[0]);
      if (url === '/__harness.html') {
        res.writeHead(200, { 'Content-Type': 'text/html; charset=utf-8' });
        res.end(HARNESS);
        return;
      }
      const file = path.join(FRONTEND, url);
      if (file.indexOf(FRONTEND) !== 0 || !fs.existsSync(file) || fs.statSync(file).isDirectory()) {
        res.writeHead(404);
        res.end('not found');
        return;
      }
      res.writeHead(200, { 'Content-Type': MIME[path.extname(file)] || 'application/octet-stream' });
      res.end(fs.readFileSync(file));
    });
    server.listen(0, '127.0.0.1', function () { resolve({ server: server, port: server.address().port
}); });
  });
}
```

```
async function launchChrome() {
  const exe = process.env.CHROME_PATH || 'C:/Program Files/Google/Chrome/Application/chrome.exe';
  if (!fs.existsSync(exe)) throw new Error('Chrome 을 찾지 못했습니다: ' + exe + ' (환경 변수 CHROME_PATH
로 지정할 수 있습니다)');
  const port = 9600 + Math.floor(Math.random() * 300);
  const dir = path.join(os.tmpdir(), 'studionx_audio_regression_' + port);
  fs.rmSync(dir, { recursive: true, force: true });
  const chrome = spawn(exe, ['--headless=new', '--remote-debugging-port=' + port, '--user-data-dir=' + d
ir, '--autoplay-policy=no-user-gesture-required', 'about:blank'], { stdio: 'ignore' });
  let target = null;
  for (let i = 0; i < 40 && !target; i++) {
    await new Promise(function (r) { setTimeout(r, 250); });
    try {
      const list = await (await fetch('http://127.0.0.1:' + port + '/json')).json();
      target = list.filter(function (x) { return x.type === 'page'; })[0] || null;
    } catch (e) { /* 아직 안 떴음 */ }
  }
  if (!target) { chrome.kill(); throw new Error('Chrome 에 연결하지 못했습니다'); }
  const ws = new WebSocket(target.webSocketDebuggerUrl);
  await new Promise(function (r) { ws.addEventListener('open', r); });
  let id = 0;
```

```
  const pending = new Map();
  ws.addEventListener('message', function (e) {
    const m = JSON.parse(e.data);
    if (m.id && pending.has(m.id)) pending.get(m.id)(m);
  });
  const send = function (method, params) {
    return new Promise(function (r) { const i = ++id; pending.set(i, r); ws.send(JSON.stringify({ id: i,
method: method, params: params || {} })); });
  };
  await send('Page.enable');
  await send('Runtime.enable');
  const version = (await send('Browser.getVersion')).result.product;
  return {
    send: send,
    version: version,
    close: function () {
      try { ws.close(); } catch (e) { /* 무시 */ }
      chrome.kill();
      // Chrome 이 아직 파일을 잡고 있을 수 있어서, 임시 폴더 지우기는 실패해도 넘어갑니다 (다음 실행이 지
우고 시작)
      try { fs.rmSync(dir, { recursive: true, force: true }); } catch (e) { /* 무시 */ }
    },
  };
}
```

```
export async function openRenderer() {
  const srv = await startServer();
  const browser = await launchChrome();
  await browser.send('Page.navigate', { url: 'http://127.0.0.1:' + srv.port + '/__harness.html' });
  for (let i = 0; i < 80; i++) {
    const r = await browser.send('Runtime.evaluate', { expression: 'window.__ready === true', returnByVa
lue: true });
    if (r.result && r.result.result && r.result.result.value === true) break;
    await new Promise(function (res) { setTimeout(res, 150); });
    if (i === 79) { browser.close(); srv.server.close(); throw new Error('하니스 페이지가 준비되지 않았습
니다'); }
  }
  return {
    version: browser.version,
    render: async function (args) {
      const r = await browser.send('Runtime.evaluate', {
        expression: 'window.__render(' + JSON.stringify(args) + ')',
        awaitPromise: true, returnByValue: true,
      });
      if (r.result && r.result.exceptionDetails) {
        const d = r.result.exceptionDetails;
        throw new Error('렌더링 실패: ' + (d.exception && d.exception.description ? d.exception.descripti
on : d.text));
      }
      const v = r.result.result.value;
      const channels = v.channels.map(function (b64) {
        const buf = Buffer.from(b64, 'base64');
        return new Float32Array(buf.buffer.slice(buf.byteOffset, buf.byteOffset + buf.byteLength));
      });
      return { channels: channels, sampleRate: v.sampleRate, frames: v.frames };
    },
```

```
    close: function () { browser.close(); srv.server.close(); },
  };
}
```

3.3 플러그인 정의

```
export function loadPluginSchemas() {
  const sql = fs.readFileSync(SEED, 'utf-8');
  const out = {};
  const re = /\(\s*'(\w+)'\s*,\s*'[^']*'\s*,\s*'(?:INSTRUMENT|EFFECT|MIDI_EFFECT)'\s*,\s*'\w+'\s*,\s
*'[^']*'\s*,\s*'(\[\{.*?\}\])'\s*,/g;
  let m;
  while ((m = re.exec(sql)) !== null) out[m[1]] = JSON.parse(m[2]);
  return out;
}
```

3.4 WAV 읽고 쓰기

```
export function writeFloatWav(file, channels, sampleRate) {
  const bytes = encodeWav(channels, sampleRate, 32, { gain: 1, dither: false });
  fs.writeFileSync(file, bytes);
  return bytes.length;
}
```

```
export function readWav(file) {
  const buf = fs.readFileSync(file);
  const v = new DataView(buf.buffer, buf.byteOffset, buf.byteLength);
  const tag = function (o) { return String.fromCharCode(buf[o], buf[o + 1], buf[o + 2], buf[o + 3]); };
  if (tag(0) !== 'RIFF' || tag(8) !== 'WAVE') throw new Error('WAV 파일이 아닙니다: ' + file);
  let pos = 12, fmt = null, dataOff = -1, dataLen = 0;
  while (pos + 8 <= buf.length) {
    const id = tag(pos), size = v.getUint32(pos + 4, true);
    if (id === 'fmt ') fmt = { format: v.getUint16(pos + 8, true), ch: v.getUint16(pos + 10, true), rat
e: v.getUint32(pos + 12, true), bits: v.getUint16(pos + 22, true) };
    if (id === 'data') { dataOff = pos + 8; dataLen = size; break; }
    pos += 8 + size + (size % 2);
  }
  if (!fmt || dataOff < 0) throw new Error('fmt 나 data 조각이 없습니다: ' + file);
  const bytes = fmt.bits / 8;
  const frames = Math.floor(dataLen / (bytes * fmt.ch));
  const channels = [];
  for (let c = 0; c < fmt.ch; c++) channels.push(new Float32Array(frames));
  for (let i = 0; i < frames; i++) {
    for (let c = 0; c < fmt.ch; c++) {
      const o = dataOff + (i * fmt.ch + c) * bytes;
      let x;
      if (fmt.format === 3 && fmt.bits === 32) x = v.getFloat32(o, true);
      else if (fmt.bits === 16) x = v.getInt16(o, true) / 32768;
      else if (fmt.bits === 24) { let n = buf[o] | (buf[o + 1] << 8) | (buf[o + 2] << 16); if (n & 0x800
000) n -= 0x1000000; x = n / 8388608; }
      else throw new Error('지원하지 않는 형식: ' + fmt.format + ' / ' + fmt.bits + 'bit');
      channels[c][i] = x;
    }
  }
```

```
  return { channels: channels, sampleRate: fmt.rate, bitDepth: fmt.bits, frames: frames };
}
```

4. 프로젝트 JSON

사람이 읽고 고치기 쉽게 마디와 박 단위로 씁니다. `buildArgs`가 앱이 받는 tick 단위(1박 960, 1마디 3840)로 바꿉니다.

항목	내용
`bpm`, `tempoChanges`	시작 템포와 템포 변경(`bar`, `bpm`, `ramp`)
`sampleRate`, `startBar`, `endBar`, `tailSec`	렌더링 범위와 꼬리
`samples`	시험용 소리를 만드는 식. 종류는 `sine`, `saw`, `kick`, `noise`, `pluck`이고 `freq`, `secs`, `amp`, `decay`, `seed`를 줍니다
`tracks`	`id`, `type`(`MIDI`, `AUDIO`), `volume_db`, `pan`, `mute`, `solo`. MASTER는 자동으로 붙고 `master_db`로 음량을 줍니다
`instruments`, `effects`	트랙 번호별 목록. `plugin`(코드), `params`(바꿀 값만), 악기 층의 `volume_db`, `pan`
`midiClips`, `audioClips`	클립의 위치(`startBar`, `lengthBars`), MIDI 노트(`startBeat`, `lengthBeat`, `pitch`, `velocity`), 오디오 클립의 `gain_db`, `loop`, `pitch`, `offsetBars`
`tolerance`	허용 오차를 프로젝트마다 덮어씀 (생략하면 기본값)

`params`에는 바꿀 값만 적습니다. 나머지는 DB 시드의 기본값이 채웁니다. 그래서 시드의 기본값이 바뀌면 이 테스트가 그 변화를 잡습니다.

5. 최신 문법으로 쓰면

```
// loadProjects 와 소리 생성 부분을 최신 문법으로 쓴 모양
export const loadProjects = (only) =>
  fs.readdirSync(PROJECTS_DIR)
    .filter((f) => f.endsWith('.json'))
    .sort()
    .map((f) => JSON.parse(fs.readFileSync(path.join(PROJECTS_DIR, f), 'utf-8')))
    .filter((p) => !only || only.includes(p.name));

const tracks = (p.tracks ?? []).map(({ id, type, volume_db = 0, pan = 0, mute, solo }) =>
  ({ track_id: id, track_type: type, volume_db, pan, is_muted: !!mute, is_solo: !!solo }));

const sleep = (ms) => new Promise((r) => setTimeout(r, ms));
```

전통 문법 (실제 코드)	최신 문법
`only.indexOf(p.name) >= 0`	`only.includes(p.name)`
`t.volume_db || 0`	구조 분해의 기본값 `volume_db = 0`
`(p.tracks || [])`	`p.tracks ?? []`
`function (r) { setTimeout(r, 250) }` 을 감싼 `Promise`	화살표 함수 `(r) => setTimeout(r, 250)`

6. 왜 이렇게

선택	이유
앱의 `renderOffline`을 그대로 부릅니다	테스트가 따로 만든 소리 계산이 아니라 사용자가 듣고 내보내는 소리와 같은 코드를 검사해야 합니다. 앱 코드가 바뀌면 테스트가 바로 그 변화를 봅니다.
서버와 DB 없이 돕니다	정적 서버만 띄워 `frontend`를 내주고, 플러그인 정의는 시드 SQL에서 읽습니다. 로그인, DB, 네트워크 문제로 테스트가 흔들리지 않고, 어느 컴퓨터에서든 Chrome만 있으면 됩니다.
플러그인 정의를 시드에서 읽습니다	실제 서버가 주는 `param_schema`와 같은 값입니다. 시드의 기본값이나 범위가 바뀌면 소리가 달라지는데, 이것도 회귀로 잡히게 합니다.
시험용 소리를 수식으로 만듭니다	바이너리 샘플 파일을 저장소에 넣지 않아도 되고, 소리의 의미(440Hz 사인, 킥, 노이즈)가 JSON에 그대로 보입니다. 노이즈는 시드를 준 선형 합동 생성기라 늘 같은 값입니다.
골든을 32bit float WAV로 저장합니다	렌더링 결과를 손실 없이 담습니다. 16bit로 줄이면 양자화 잡음이 비교 지표에 섞입니다. 앱이 내보내는 `encodeWav`를 그대로 써서, 파일 형식도 앱의 출력과 같습니다.
프로젝트를 5개로, 길이를 2마디(약 4초)로 했습니다	골든 5개가 약 8MB입니다. 길게 만들면 저장소가 빠르게 커지고, 짧아도 악기, 이펙트, 믹서, 템포 변경의 경로를 모두 지나갑니다.
임시 폴더 삭제 실패는 넘어갑니다	Windows에서 Chrome을 끈 직후에는 파일이 잠겨 있어 삭제가 거부됩니다(처음 실행에서 `EPERM`이 나서 알게 됨). 임시 폴더는 다음 실행이 지우고 시작하므로 남아도 해가 없습니다.
Chrome 위치는 환경 변수로 바꿀 수 있습니다	컴퓨터마다 설치 위치가 다를 수 있습니다. 기본은 이 프로젝트가 쓰는 경로입니다.

7. 시험

- `selftest.mjs`의 WAV 시험: 32bit float 스테레오를 쓰고 읽으면 한 비트도 틀리지 않고 돌아옵니다(채널 수, 프레임 수, 샘플레이트, 비트 깊이 포함).
- 프로젝트 5개가 모두 변환과 렌더링을 마치고 골든과 통과하며, 샘플레이트와 프레임 수가 골든과 같습니다.
- 재현성: 같은 프로젝트를 다시 렌더링하면 최대 차이가 1e-6 이하입니다. 실제로는 0에서 약 1.8e-7 사이였습니다(32bit float의 반올림 한계).
- 시험 중 발견한 일: 처음 실행에서 렌더링은 끝났는데 Chrome을 닫는 순간 임시 폴더 삭제가 `EPERM`으로 거부되어 프로그램이 오류로 끝났습니다. 렌더링 결과는 이미 받았으므로 삭제 실패를 무시하도록 고쳤습니다.